Lab 12 — Pods, Deployments and probes

Theory — how to run an application in a cluster without ever creating a Pod by hand: replicas, labels, an update that cuts nothing, a rollback in one command, and probes that tell the cluster what "ready" means.

Objectives

- Understand the **Deployment → ReplicaSet → Pod** chain and the role of labels and selectors.
- Scale, update (*rolling update*) and roll back (`rollout undo`).
- Tell the three probes apart — readiness, liveness, startup — and their effects.
- Read a stuck rollout and know why the old Pods keep serving.
- Relate all of this to the `HEALTHCHECK` and `--restart` of the Docker labs.

---

1. Nobody creates a Pod by hand

In lab 11, a deleted Pod did not come back, but a Pod under a Deployment reappeared. The reason lies in three nested objects:

Object	What it describes	Who creates it
Pod	one or more containers running together	the ReplicaSet
ReplicaSet	"N copies of this Pod model must exist"	the Deployment
Deployment	"here is the Pod model, the number of replicas, and how to move from one version to the next"	**you**

The Deployment carries a Pod **template** (`spec.template`) and a number of replicas. On every change of the template (new image, new variable), it creates a **new** ReplicaSet and transfers the replicas from the old one to the new one, progressively. Old ReplicaSets remain at `0` replicas: that is the history that makes rollback possible. You never touch a ReplicaSet directly; you only create a lone Pod for a test.

> **→ DOCKER RECAP**
> A Deployment is `podman run` plus `--restart=always` plus `--scale` plus a zero-downtime update script, written once and for all in an object the cluster executes. The Pod template holds what you used to put on the `podman run` line: image, ports, variables, volumes. The template's `restartPolicy: Always` takes over the `--restart` of lab 03 — enforced by the kubelet, which never goes away.

2. Labels and selectors: how objects find each other

A ReplicaSet does not "contain" its Pods: it **selects** them by their labels. The Deployment declares `selector.matchLabels: {app: api}` and its template carries `labels: {app: api}`; every Pod bearing that label and belonging to the ReplicaSet counts as a replica. The cluster adds a `pod-template-hash` label to every Pod, computed from the template: that is what distinguishes the version 1.0 Pods from the 1.1 ones, and what gives the names their shape `api-6c7bb54fc5-2th65` (Deployment name, template hash, random suffix).

Labels are the universal mechanism of Kubernetes: Services (lab 13) pick their Pods the same way, and `kubectl get pods -l app=api` filters as a controller would. A label is a free key/value pair; company conventions fix a few (`app`, `version`, `tier`, `environment`), often under the `app.kubernetes.io/` prefix.

3. Scaling and updating

`kubectl scale deployment api --replicas=3` changes one number in the spec; the ReplicaSet creates or deletes Pods to reach it. In production, that number is set in the manifest (and changed through `apply`), or driven by a HorizontalPodAutoscaler.

The update is the most important operation. `kubectl set image deployment/api api=localhost/api:1.1` (or an `apply` of the modified manifest) triggers a **rolling update**: the Deployment creates a ReplicaSet for the new version, adds one Pod to it, waits until it is **ready**, removes one old Pod, and repeats. Two settings in `strategy.rollingUpdate` bound the movement: `maxSurge` (how many Pods above the desired number, 25 % by default) and `maxUnavailable` (how many may be missing, 25 % by default). With three replicas, the cluster never drops below two ready Pods, and the service does not cut out.

`kubectl rollout status` follows the operation, `rollout history` lists the revisions, `rollout undo` returns to the previous one (or `--to-revision=N`). Each revision is a ReplicaSet kept at zero; `revisionHistoryLimit` (10 by default) bounds their number.

> **REMEMBER**
> A failing rollout is not an outage: if the new Pods never become ready, the Deployment **stops** after the first one (or the first `maxSurge` ones) and lets the old ones serve. `kubectl rollout status` times out, `kubectl get deployment` shows `UP-TO-DATE 1 / AVAILABLE 3`, and `rollout undo` brings everything back within seconds. That is what `compose up` cannot do: it replaces, and you discover the outage afterwards.

4. Probes: telling the cluster what "ready" and "alive" mean

The rolling update rests on one notion: a **ready** Pod. Without a probe, the cluster considers a container ready as soon as its process has started — for a JVM, ten seconds before it accepts a request. **Probes** fix that. There are three, with three different effects:

Probe	Question asked	If it fails	Use
readinessProbe	"can you take traffic?"	the Pod is removed from the Services, **not restarted**; `READY 0/1`	slow start-up, temporarily missing dependency
livenessProbe	"are you still alive?"	the container is **killed and restarted** (`RESTARTS` increases)	deadlock, memory leak, infinite loop
startupProbe	"have you finished starting?"	the other two are suspended until it succeeds	applications that start very slowly

Each probe is a command (`exec`), an HTTP request (`httpGet`) or a TCP connection (`tcpSocket`), with `initialDelaySeconds`, `periodSeconds`, `failureThreshold`. They are run by the **kubelet**, from the node, with nothing installed in the image — that is the advantage over the `HEALTHCHECK` of lab 10, which required `curl` or `wget` inside the container.

→ SPRING BOOT
Spring Boot exposes two endpoints made for this: `/actuator/health/readiness` (is the application ready to serve: connection pool open, caches loaded) and `/actuator/health/liveness` (is the application in a healthy state). Enabled by `management.endpoint.health.probes.enabled=true` (automatic when Spring detects Kubernetes), they reflect the internal state (`ApplicationAvailability`) rather than a mere "the port answers". Our fake API only has `/actuator/health`; this lab's probes use it for both.

PITFALL
A badly tuned **liveness** is more dangerous than no probe at all. If it tests a dependency (the database), a database outage restarts the API in a loop, which helps nothing. If its delay is too short, a slow JVM is killed before it has started, then again, then `CrashLoopBackOff`. And above all: a liveness that fails on the **new** version does not block the rollout — the new Pods become `Ready` before the liveness kills them, the old ones are removed, and **the whole service** goes down. Readiness protects the deployment; liveness protects against deadlocks. Do not confuse their roles.

5. Reading a Pod's state

`kubectl get pods` sums everything up in three columns: `READY` (ready containers / total), `STATUS`, `RESTARTS`. The states to recognise:

`STATUS`	Meaning	First reflex
`Pending`	not placed yet, or waiting for an image/volume	`describe` → Events
`ContainerCreating`	image being downloaded, volumes being mounted	wait, then `describe`
`Running` `0/1`	runs but not ready: readiness failing	`describe` → `Readiness probe failed`
`Error` / `CrashLoopBackOff`	the process exits with an error, relaunched with a growing delay	`kubectl logs --previous`
`ImagePullBackOff`	image not found (lab 11)	name, tag, import
`Terminating`	being stopped (`SIGTERM`, then `SIGKILL` after `terminationGracePeriodSeconds`, 30 s)	normal during a rollout

`CrashLoopBackOff` deserves a precision: it is not a state of the application but of the **kubelet**, which waits longer and longer (10 s, 20 s, 40 s… up to 5 min) before relaunching a container that keeps exiting. `kubectl logs pod --previous` shows the output of the previous attempt: that is where the cause is.

K3S
Nothing k3s-specific in this lab: Deployments, ReplicaSets and probes are the core of the API, identical everywhere. The only visible difference is **speed**: on a single node, the images are already there and a three-replica rollout takes ten seconds; on a managed cluster, each new Pod may wait for a node, a pull from the registry and a load balancer, and the same rollout takes a few minutes.

6. In the workplace

- **Every service is a Deployment** (or a StatefulSet for databases, lab 15). The manifest is versioned, and a production release is an `apply` with a new image — by digest, lab 10.

- **Two replicas minimum** in production, so that a rollout or a node failure cuts nothing; three for critical services.
- **Readiness mandatory, liveness considered**: readiness on `/actuator/health/readiness`, liveness on `/actuator/health/liveness` with a generous delay, never on a dependency.
- **Rollback is a reflex, not a drama**: `rollout undo` is part of the incident procedure, and `rollout history` of the post-incident review.
- **Resources** (`requests`/`limits`, lab 16) complete the template: without them, the scheduler places blindly.

Remember

- Deployment → ReplicaSet → Pod: you write the first, the cluster creates the other two; labels link them.
- `scale` changes a number; `set image` or `apply` of a new template triggers a rolling update bounded by `maxSurge` and `maxUnavailable`.
- A rollout whose Pods do not become ready stops and lets the old ones serve; `rollout undo` goes back.
- Readiness = remove from traffic without restarting; liveness = restart; startup = wait before judging.
- A badly tuned liveness kills a healthy service; a failing readiness on the new version blocks the rollout harmlessly.
- `CrashLoopBackOff` is a kubelet delay; the cause is in `kubectl logs --previous`.
- Probes are run by the kubelet, with no tool in the image — unlike the Docker `HEALTHCHECK`.

Vocabulary

Deployment: object describing a Pod template, a number of replicas and an update strategy. — **ReplicaSet**: controller maintaining N identical Pods; created by the Deployment. — **template**: the Pod model in `spec.template`. — **label / selector**: key/value tag / filter on labels. — **`pod-template-hash`**: computed label identifying a version of the template. — **rolling update**: progressive replacement of the Pods; **`maxSurge`** / **`maxUnavailable`**: its bounds. — **revision**: a ReplicaSet kept for `rollout undo`. — **readiness / liveness / startup probe**: kubelet probes and their effects. — **`CrashLoopBackOff`**: growing delay before relaunching a container that exits with an error. — **`terminationGracePeriodSeconds`**: delay between `SIGTERM` and `SIGKILL`.